



Software Engineering Practice

Analysis and Design of a PoS System

Name: Wang Hao (汪浩)

NUIST ID: 202383890022

SETU ID: 20109992

Email: 1551921411@qq.com

GitHub URL: https://github.com/denniswang12345/ProjectPos_2026

Demo Video URL: To be added after recording

10 / 19 / 2026

Contents

1 Introduction and scope 3

2 Business rules and project assumptions 4

3 Product backlog and prioritization 5

4 Proposed sprint plan and working agreements 6

5 Burndown plan and progress recording 7

6 Use case overview 8

7 Process Sale fully dressed use case 9

8 Process Sale extensions and operation contracts 10

9 Handle Returns fully dressed use case 11

10 Handle Returns extensions and operation contracts 12

11 System sequence diagrams 13

12 Domain model 14

13 Logical architecture and package dependencies 15

14 Object design for Process Sale 16

15 Object design for Handle Returns 17

16 Design class model and responsibility mapping 18

17 Implementation and running the application 19

18 Demonstration evidence 20

19 Verification and traceability 21

20 Git evidence and submission completion 22

21 References 23

1 Introduction and scope

This project applies iterative planning and object-oriented analysis and design to a small supermarket point-of-sale system. It supports Process Sale and Handle Returns. A runnable Python console application demonstrates cash sales, original-receipt returns and persistent inventory updates. The same use cases drive the system sequence diagrams, domain model, interaction diagrams and implementation.

The Process Sale design adapts the NextGen POS example in Craig Larman, Applying UML and Patterns, third edition, especially Chapter 6 section 6.8 and Chapters 9, 10, 13, 17 and 18. Handle Returns develops the brief return scenario in Chapter 6 section 6.2 into a corresponding complete design. The project deliberately limits the textbook scenario to the cash-only scope in the assignment.

The current contributor is Wang Hao (汪浩), NUIST ID 202383890022, SETU ID 20109992. The work is individual. The assignment suggests teams of two or three, so acceptance of individual submission should be confirmed with the instructor. No other team members or collaborative activities are claimed.

The repository URL is https://github.com/denniswang12345/ProjectPos_2026. A presentation video has not yet been recorded or uploaded. Its final bilibili URL must be added to the cover before submission. The report describes verified local behavior and distinguishes the proposed sprint plan from historical project evidence.

Deliverable	Coverage
Agile planning	Prioritized stories, two proposed sprints, tasks, planned burndown
Requirements and UML	Two fully dressed use cases, use case diagram, two SSDs, domain and design class diagrams, package diagram and two design interactions
Implementation	Python 3.10+ console application, SQLite persistence, 18 automated tests
Presentation	PowerPoint and separate recording script
Quiz work	Answers to all 15 questions in each quiz

2 Business rules and project assumptions

BR1 Currency is CNY. Catalog prices are treated as tax-inclusive final prices. Money is stored as integer cents and input uses Decimal validation. Separate tax calculation, promotions, credit cards and external accounting integration are outside this teaching implementation.

BR2 Only positive integer quantities are accepted. Unknown product identifiers are rejected. Stock is validated again when payment commits, and no committed sale may make stock negative. Scanning and cancelling a basket do not change stock.

BR3 A completed cash sale stores the original unit price, quantity, date, amount tendered and total. Change equals amount tendered minus total. Underpayment leaves the sale awaiting payment.

BR4 Returns require a completed original receipt in the local database and must occur within 30 calendar days, inclusive. This return period is a project policy assumption, not a quoted assignment rule or a legal claim. Missing-receipt and expired returns are rejected rather than overridden.

BR5 For every original sale line, cumulative returned quantity must remain between zero and the purchased quantity. Refund value uses the stored original unit price. A later catalog price change does not change the refund.

BR6 The cashier inspects returned goods and chooses whether they are resalable. Only resalable goods increase saleable inventory. A damaged accepted return still records its refund without increasing saleable stock.

BR7 Sale, payment record and stock decrement commit as one SQLite transaction. A return, refund record, cumulative returned quantities and eligible restocking likewise commit together. An error rolls back all changes. An operation token uniquely identifies a confirmed request. An exact replay returns the existing receipt; reuse for different request content is rejected.

BR8 The demonstration assumes an identified cashier using a single terminal. It does not implement authentication, access control, a physical cash drawer, a printer driver or a customer display. A receipt is displayed on the console and retained as database records. The cashier physically hands over change or refund cash.

3 Product backlog and prioritization

The proposed backlog contains 32 story points. Story points express relative complexity and uncertainty, not hours. Priority is top to bottom, starting with a complete cash-sale increment before adding returns. Return eligibility and historical price preservation are higher priority than presentation polish.

For an individual project, Wang Hao performs product-owner, developer and facilitation responsibilities as separate activities. This is a Scrum-inspired learning plan rather than a claim that one person forms a complete Scrum team. Feedback from the instructor or a peer is planned for each sprint review.

ID	User story and acceptance condition	SP	Sprint
US1	As a cashier, I can scan products and quantities to see an accurate basket and running total. Unknown items and nonpositive quantities are rejected.	5	1
US2	As a cashier, I can accept sufficient cash and obtain correct change. Underpayment records no sale or stock movement.	5	1
US3	As a store owner, I can keep stock consistent with completed sales. Insufficient stock rolls back the complete sale.	3	1
US4	As a customer, I receive a receipt identifying what I bought and the amount paid.	2	1
US5	As a cashier, I can retrieve an original sale and record a partial or full return.	5	2
US6	As a store owner, I can reject missing receipts, expired returns and quantities beyond the unreturned amount.	3	2
US7	As a customer, I receive the original-price cash refund, with eligible goods returned to stock.	3	2
US8	As a cashier, I can retry a confirmed operation without duplicate sale or refund records.	2	2
US9	As a reviewer, I can trace requirements through UML models and tests in the report.	3	2
US10	As a presenter, I can demonstrate both use cases with the slide deck and program.	1	2

4 Proposed sprint plan and working agreements

Two three-day sprints are proposed, using relative project days rather than invented historical dates. Sprint 1 has 15 points and delivers a cash sale. Sprint 2 has 17 points and adds safe returns plus the submission materials. The plan should be re-estimated after Sprint 1 based on measured progress.

Sprint 1 goal: a cashier can record a basket, take cash, print a console receipt and persist the stock change. Day 1: refine UC1, sketch the domain model and create the product catalog and basket. Day 2: implement totals, cash validation and transaction persistence. Day 3: exercise rollback and persistence tests, review the cash-sale demo and revise the backlog.

Sprint 2 goal: a cashier can refund eligible items without over-refunding or corrupting stock. Day 1: refine UC2 and implement original-sale lookup and eligibility. Day 2: implement original-price refunds, restocking, token replay and failure tests. Day 3: review both scenarios, reconcile UML with code, finalize the report and slides, and record the demonstration.

At the start of each sprint, select stories and identify the main risks. Hold a brief daily self-review of the goal, progress, obstacles and the next concrete task. At review, demonstrate working behavior to a peer or instructor and record actual feedback. At retrospective, choose one improvement for the next sprint.

Definition of Ready: a story has a clear actor, business value, testable acceptance criteria and understood dependencies. Definition of Done: its behavior works, relevant tests pass, state changes persist or roll back correctly, diagrams match the design, and the change is committed with a meaningful message.

Change handling: new requests enter the backlog. Scope may be negotiated during a sprint if the goal remains achievable. Review findings can change the next sprint. No retrospective feedback or review attendance is claimed until it actually occurs.

5 Burndown plan and progress recording

A burndown chart tracks remaining work across a sprint timebox. This figure shows only ideal planned lines: Sprint 1 starts at 15 points and Sprint 2 at 17. It is not a record of six elapsed project days. The assignment permits a project plan; actual remaining work should be added daily if the instructor requires execution evidence.

Credit a story only when it meets the Definition of Done. Measure remaining points at the same time each day. Do not reduce the actual line merely because time has passed. If scope changes, annotate the change and replan transparently.

Sprint day	Sprint 1 ideal SP	Sprint 1 actual SP	Sprint 2 ideal SP	Sprint 2 actual SP
0	15.00	Not recorded	17.00	Not recorded
1	10.00	Not recorded	11.33	Not recorded
2	5.00	Not recorded	5.67	Not recorded
3	0.00	Not recorded	0.00	Not recorded

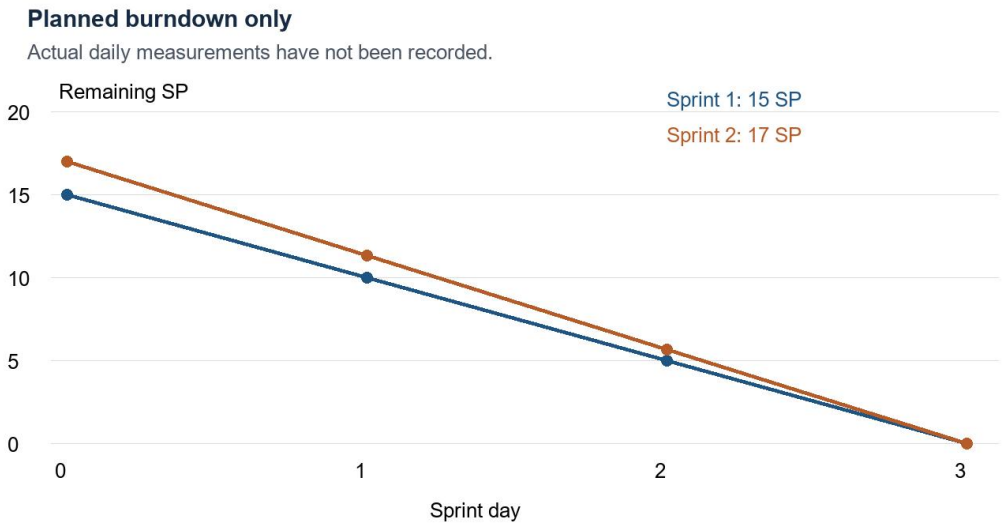


Figure 1 Proposed ideal burndown for two three-day sprints

6 Use case overview

The system boundary contains the two user-goal use cases Process Sale and Handle Returns. The cashier operates the POS. The customer supplies goods and cash to the cashier and receives goods, cash and receipts. Because the customer does not directly use this console, the diagram shows only the cashier as a directly interacting actor.

The cashier actor remains a business role even when the same student develops and demonstrates the software. The database is an internal implementation component rather than an external use-case actor. No payment authorization service is needed for the selected cash-only scope.

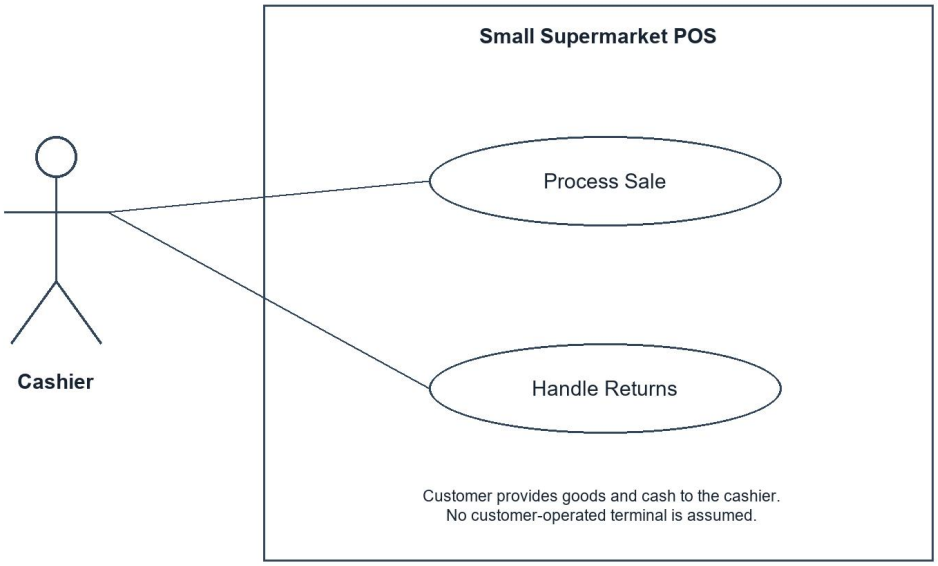


Figure 2 Use case diagram for the selected system boundary

7 Process Sale fully dressed use case

Use case UC1 Process Sale. Scope: small supermarket POS. Level: user goal. Primary actor: Cashier. Trigger: a customer presents goods for purchase.

Stakeholders and interests: the cashier needs fast and accurate recording, the customer needs a correct total and change plus proof of purchase, and the store needs reliable sale and stock records.

Preconditions: an identified cashier is using the training terminal; the catalog and local database are available. Success guarantee: one completed sale records original prices, payment and change, stock decreases exactly once, and the receipt can be retrieved. Minimal guarantee: no failed or cancelled attempt changes persistent sales or inventory.

Main success scenario
1. The customer presents goods at checkout.
2. The cashier starts a new sale.
3. The cashier enters a product identifier and a positive quantity.
4. The system records the line and shows its description, price and running total. Steps 3 and 4 repeat for the remaining products.
5. The cashier ends item entry. The system presents the final total.
6. The customer gives cash and the cashier enters the amount tendered.
7. The system validates payment and stock, saves the sale and payment and decreases stock in one transaction.
8. The system displays the receipt and change.
9. The cashier hands over change, receipt information and goods.

Special requirements: amounts must be exact to cents; failed persistence must leave no partial update; the receipt must uniquely identify the sale and preserve historical prices. Display responsiveness is a usability target, not a measured performance claim.

Technology and data variations: the teaching UI accepts typed product codes and integer quantities. A barcode scanner could supply the same code later. Frequency: expected to be the principal checkout activity; no measured transaction volume is assumed. Open issues: production authentication, physical receipt printing and tax integration.

8 Process Sale extensions and operation contracts

UC1 extensions
3a. Unknown product code: show an error without changing the basket. The cashier corrects the code.
3b. Nonpositive or noninteger quantity: reject the entry.
5a. Empty basket: reject completion.
6a. Invalid amount or insufficient cash: report the problem and keep the ended sale available for payment retry.
7a. Insufficient stock or a database error: roll back all sale and inventory changes. The cashier cancels or corrects the purchase.
7b. Exact replay of a confirmed request token: return the saved receipt without applying stock changes again. A conflicting token request is rejected.
Before payment confirmation, the cashier may cancel the sale without a stock change.

Operation makeNewSale(): precondition: no unpaid sale is active. Postcondition: create an empty Sale in OPEN state. Operation enterItem(sku, quantity): precondition: an OPEN sale and a valid product. Postcondition: create or increase its SalesLineItem and expose the new running total.

Operation endSale(): precondition: an OPEN, nonempty sale. Postcondition: set its state to ENDED and expose the total. Operation makeCashPayment(amount, token): precondition: an ENDED sale, valid token and sufficient cash and stock. Postcondition on a first successful request: persist one sale and its original-price lines, payment and token receipt; decrease stock; set PAID; return change. An exact replay returns its prior result.

9 Handle Returns fully dressed use case

Use case UC2 Handle Returns. Scope: small supermarket POS. Level: user goal. Primary actor: Cashier. Trigger: a customer presents previously purchased goods for return.

Stakeholders and interests: the customer needs the correct refund, the cashier needs clear eligibility and refund information, and the store needs protection against over-refunding and incorrect stock increases.

Preconditions: an identified cashier uses the terminal and the original completed cash sale exists in the local database. Success guarantee: one return references the original sale, records original-price refund lines, increases cumulative returned quantities, restocks eligible goods and provides a refund receipt. Minimal guarantee: a rejected, cancelled or failed return changes no persistent record.

Main success scenario
1. The customer presents goods and the original receipt.
2. The cashier enters the receipt identifier.
3. The system retrieves original lines, unit prices and remaining returnable quantities.
4. The cashier inspects the goods and enters each returned product, quantity and restock decision.
5. The cashier confirms the cash refund request.
6. The system checks the 30-day policy and each quantity against the original sale less all prior returns, then calculates the refund using original prices.
7. The system saves the return and refund, updates cumulative returned quantities and increases stock only for eligible goods in one transaction.
8. The system displays a refund receipt and amount.
9. The cashier gives the customer that amount in cash and provides the receipt information.

Special requirements: historical prices must survive catalog changes; repeated confirmed requests must not issue a second recorded refund; all changes must be atomic. The program records the cash refund; handing over physical cash is the cashier responsibility.

Technology and data variations: multiple selected products can be returned together; partial and full returns use the same flow. Frequency: lower than sales but not quantitatively estimated. Open issues: managerial exceptions, legal policy configuration, noncash refunds and damaged-stock disposition.

10 Handle Returns extensions and operation contracts

UC2 extensions
2a. Missing or unknown receipt: reject the return.
4a. Product absent from original sale, repeated SKU entry, or invalid quantity: reject the complete request.
4b. Goods are damaged: the cashier may accept the refund with restock false.
5a. The cashier declines confirmation: discard the uncommitted selection.
6a. Original sale is older than 30 days or dated in the future: reject the return.
6b. Requested quantity exceeds the purchased quantity minus cumulative returns: reject without recording a refund.
7a. Database write fails: roll back all return and stock changes.
7b. Exact confirmed request replay: return the saved refund receipt. A conflicting use of the token is rejected.

Operation findSale(receiptId): precondition: a valid identifier has been entered. Postcondition: return original sale date, original line prices and purchased and already-returned quantities, without changing state.

Operation completeReturn(receiptId, items, token): precondition: at least one selected item, positive integer quantities, an existing completed sale within the policy window and sufficient remaining allowance. Postcondition on success: create one Return and ReturnLineItems; record its cash-refund total; increase original-line returned quantities; restock only selected eligible quantities; persist the token and receipt. All persistent changes are committed together.

The SSD treats the POS as a black box. Its system events do not expose SQL statements or internal objects. The console collects a temporary return selection locally and submits it as one request, so the SSD has no separate startReturn or endReturn events.

11 System sequence diagrams

Each SSD shows the cashier and the POS as a single black box. The sale scenario repeats item entry before accepting cash. The return scenario retrieves the original receipt and submits a confirmed item selection. Internal objects and SQL belong to the design interactions instead.

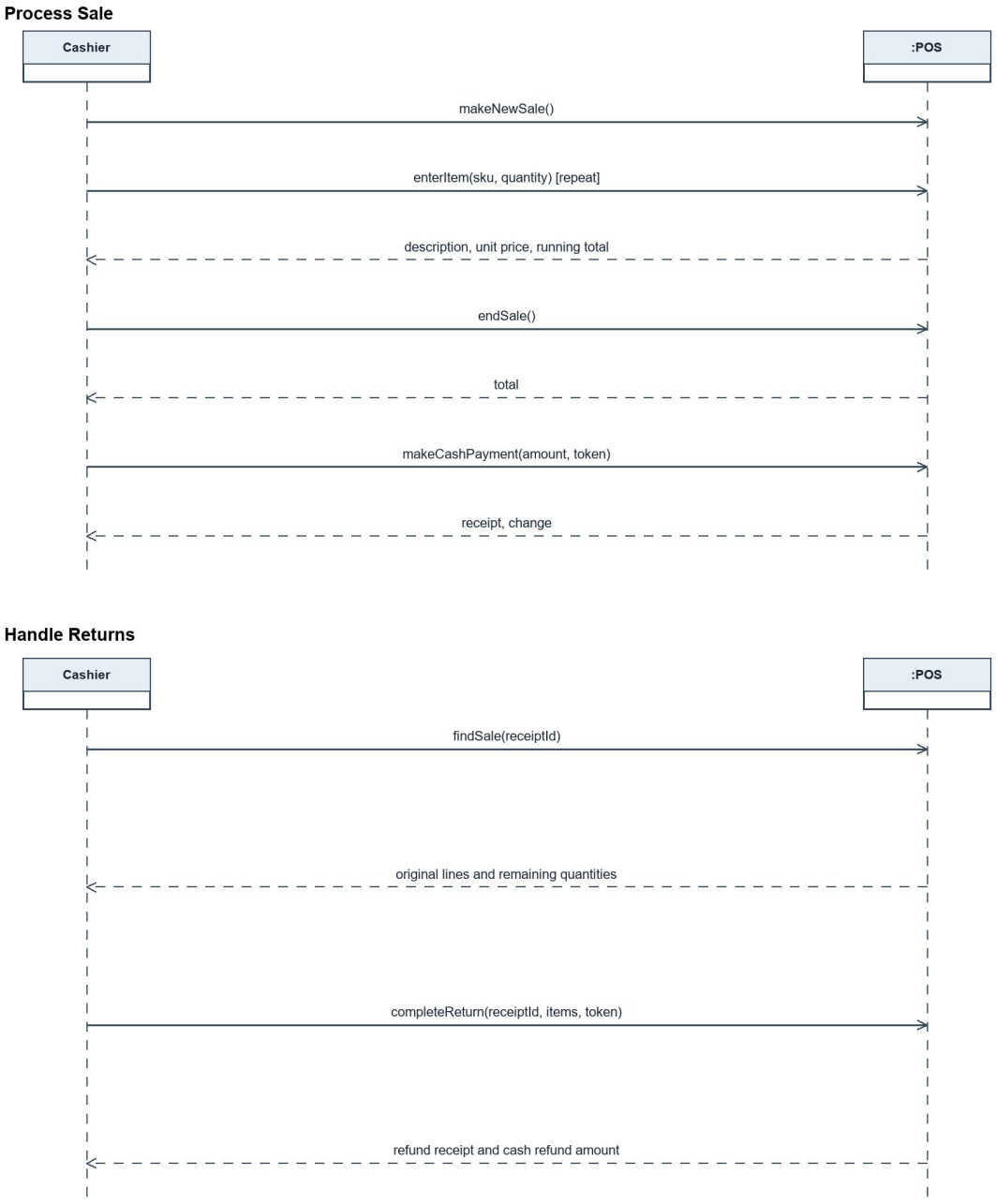


Figure 3 and Figure 4 System sequence diagrams for sale and return

12 Domain model

The domain model identifies business concepts independently of Python methods or database tables. A Sale contains one or more SalesLineItems once completed. Each line describes a purchased product and preserves the original price and returned quantity. ProductDescription describes products shared by sale lines. InventoryItem tracks their current saleable quantity.

A Return relates to one original Sale and contains one or more ReturnLineItems. Each returned line references an original SalesLineItem. CashPayment and CashRefund represent the business cash exchanges. They are concepts in this analysis model; the implementation stores their amounts within sale and return records rather than introducing separate payment classes.

The maximum number of returns per sale is unconstrained, while their combined quantities are constrained by the original purchase. A product description is retained independently of any individual sale so that repeated sales do not duplicate catalog data. Original prices remain in sale and return lines, since current product prices can change.

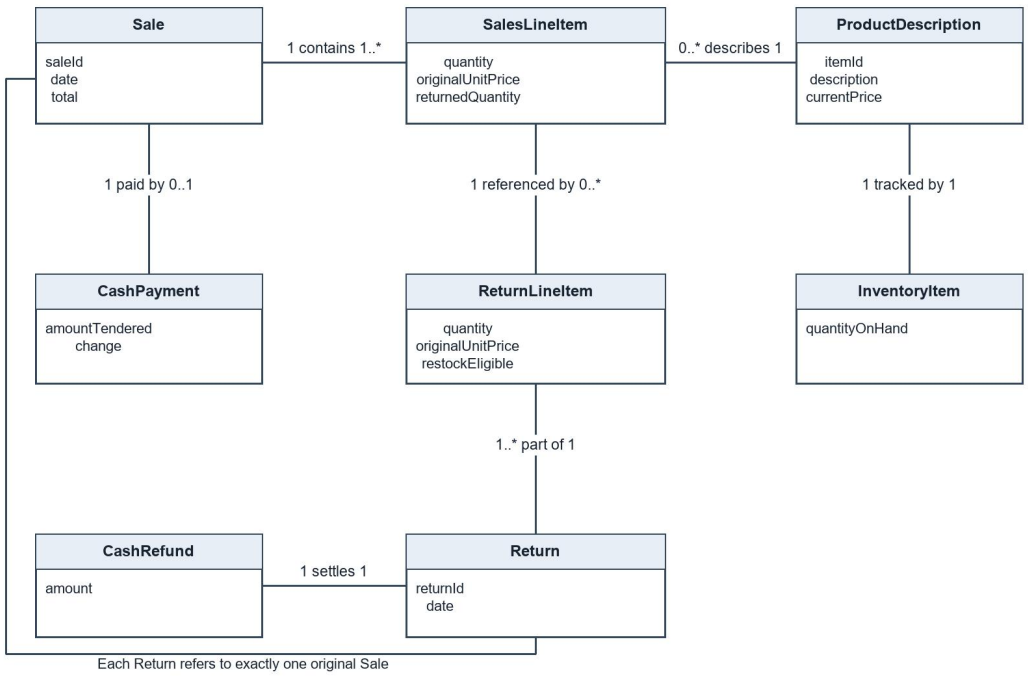


Figure 5 Domain model with association multiplicities

13 Logical architecture and package dependencies

The logical design separates presentation, application coordination, domain calculations and persistence. Presentation parses console commands and formats output. PosController coordinates a user request, validates transaction-wide rules and manages the transaction boundary. Domain objects calculate line subtotals and aggregate totals. SQLiteRepository provides the connection, schema and catalog access.

The console depends on PosController. The controller uses domain objects and SQLiteRepository. Domain objects do not call the UI or database. Persistence uses the Python sqlite3 standard-library driver and an embedded database. These dependencies support model-view separation: rules are callable by tests without console interaction.

The figure shows logical packages. In this deliberately small implementation, these components are colocated in pos.py rather than separate physical directories. The controller currently uses repository.db for transaction SQL. This is a documented coupling tradeoff; a larger application should move commit workflows behind repository methods or a unit-of-work abstraction.

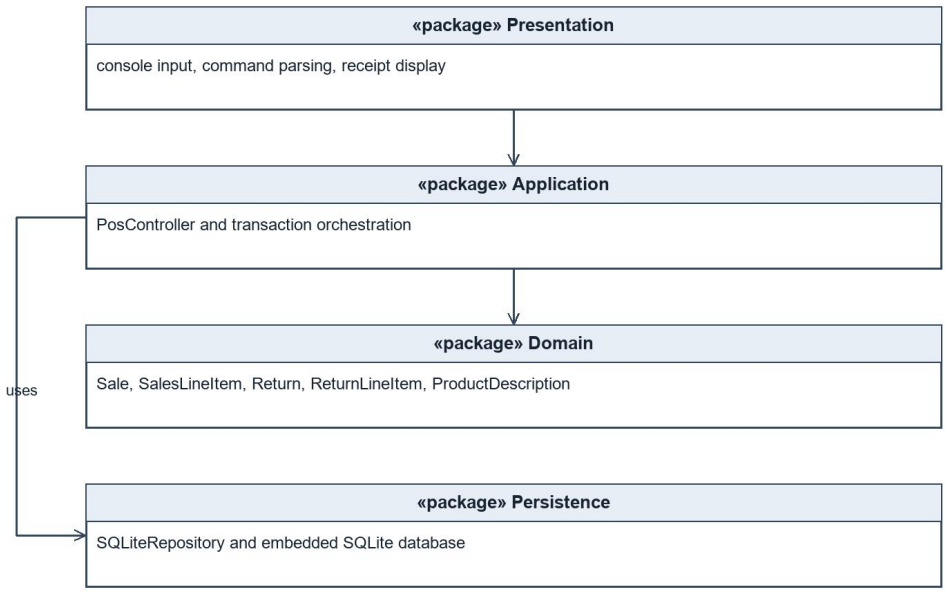


Figure 6 UML package diagram for logical layers

14 Object design for Process Sale

This interaction refines the sale SSD into collaborations. The console delegates system events to PosController, which acts as the GRASP Controller. It looks up a ProductDescription and asks the current Sale to add an item. Sale creates a SalesLineItem when a product first appears, following Creator because it contains the lines.

SalesLineItem knows its unit price and quantity and calculates its own subtotal. Sale knows its lines and calculates their sum. These assignments follow Information Expert and avoid placing arithmetic in the user interface.

Payment coordinates persistence and inventory as a single transaction. The diagram uses summarized SQL messages rather than claiming that the repository has undocumented save methods. The controller sets the Sale state to ENDED or PAID directly in this small design; these transitions could become explicit domain methods in a larger version.

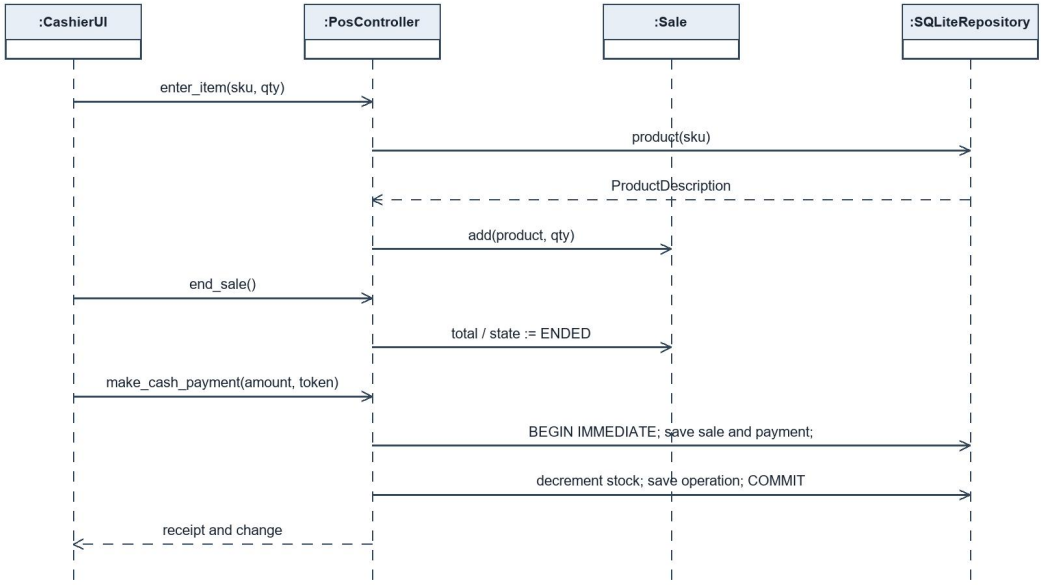


Figure 7 Design interaction for item entry and cash payment

15 Object design for Handle Returns

PosController receives the confirmed return selection. It first checks the operation token inside BEGIN IMMEDIATE, then retrieves the original sale and line state. An exact prior request returns its receipt without running the remaining creation and stock steps.

The controller validates policy and quantities, creates ReturnLineItems at stored original prices, and creates a Return that sums their subtotals. A short-lived Return aggregate keeps refund arithmetic independent of console output. SQL then records the return and increments original-line returned quantities and eligible inventory quantities.

The write lock serializes these checks and updates across SQLite connections so two competing requests cannot both consume the same remaining quantity. Transactions prevent partial state. Automated tests cover sequential replay and rollback; a dedicated multi-client concurrency stress test is not claimed.

No Factory or Adapter class is added merely to display a pattern. A future payment or inventory service could justify an adapter. The present system has one embedded database and one cash-payment method.

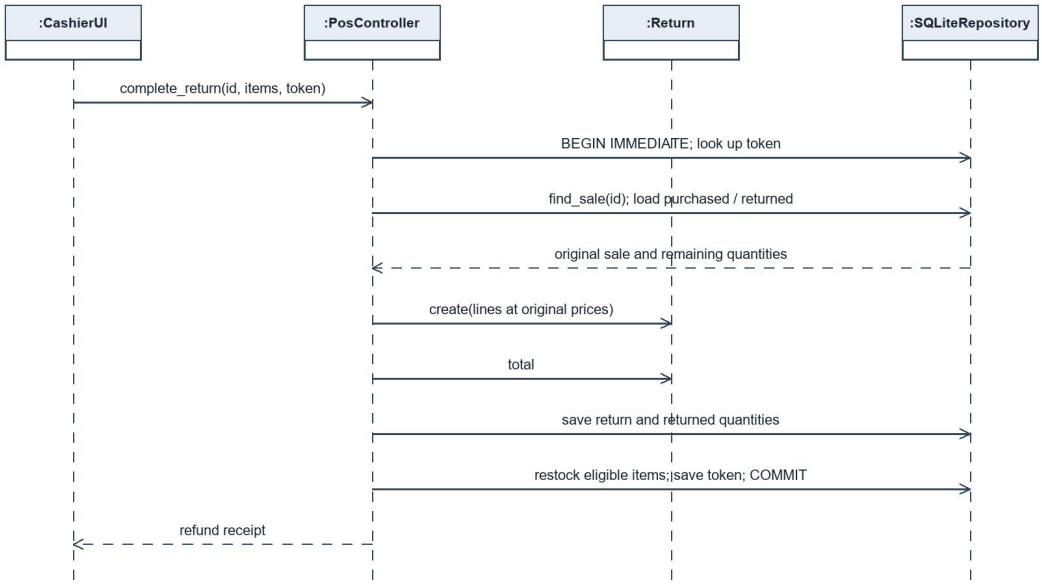


Figure 8 Design interaction for a confirmed cash return

16 Design class model and responsibility mapping

The design model adds software classes, types and callable operations. PosController holds one SQLiteRepository and an optional current Sale. Sale owns zero or more lines while open and requires at least one before completion. Each SalesLineItem references an immutable ProductDescription. Return owns selected ReturnLineItems.

The design follows the implementation class names. CashierUI is the console function interactive(), not a Python class. Inventory and cash payment/refund records are represented through SQLite columns. Those choices explain why the design class model differs from the conceptual domain model.

Low coupling is achieved between the UI and domain calculations. High cohesion is strongest in line and total calculation. PosController also carries persistence orchestration and return-policy logic, which is acceptable for this bounded teaching system but would merit separation as requirements grow.

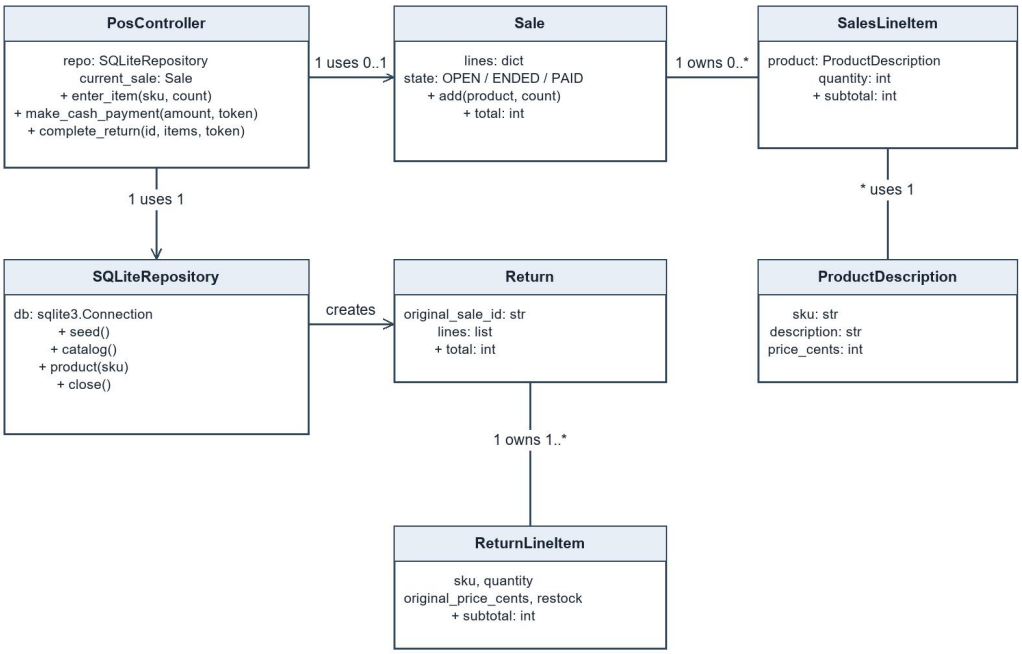


Figure 9 Design class diagram aligned with pos.py

17 Implementation and running the application

The implementation uses Python 3.10 or newer and only standard-library modules. Run python pos.py --demo for an isolated demonstration that does not write a database. Run python pos.py for the persistent console; it creates pos.db in the current folder and seeds three products only if they are absent. An alternate database can be selected with --db PATH.

The console supports catalog, new, add SKU QTY, total, pay AMOUNT, cancel, find RECEIPT, return and quit. For a sale, enter new, add MILK 2, add BREAD 1, total and pay 30.00. Copy the resulting receipt ID when trying the return command. Enter MILK 1 yes and then done to select one resalable milk item, then confirm the cash refund.

Tables are products, sales, sale_lines, returns, return_lines and operations. Primary keys identify transactions and original sale lines. Foreign keys preserve references. CHECK constraints keep stock nonnegative and cumulative returns within purchased quantities. BEGIN IMMEDIATE and explicit COMMIT or ROLLBACK protect the complete write workflow.

All application code is supplied in ProjectPos_2026/pos.py, with tests in tests/test_pos.py. Source files are provided in the accompanying project archive rather than duplicated as a long report listing. The fixed examples use integer-cent arithmetic for both sale totals and refunds.

Limitations: the console is a training interface; the physical cash drawer balance is not tracked; authentication, electronic payment, printer recovery, distributed deployment and production audit permissions are outside scope. A failed screen display after a successful commit does not mean payment failed: receipt data can be retrieved from persistent records.

18 Demonstration evidence

The accompanying image reproduces text captured from an actual local execution of python pos.py --demo. It is an execution transcript image rather than a screenshot of a graphical POS terminal. Generated transaction identifiers vary on every run.

The sale contains MILK quantity 2 at CNY 6.50 and BREAD quantity 1 at CNY 8.00. Total is CNY 21.00, cash tendered is CNY 30.00 and change is CNY 9.00. Milk stock decreases from 20 to 18 and bread stock from 15 to 14.

Returning one resalable milk item refunds CNY 6.50 and raises milk stock to 19. Retrying the same token returns the same refund receipt. Attempting to return two additional milk items fails because only one remains returnable. The live recording should show the console commands and explain these outcomes.

```
DEMO / isolated in-memory data / currency CNY
Initial stock [('APPLE', 'Apple pack', 1200, 10), ('BREAD', 'Bread', 800, 15), ('MILK', 'Milk 1L', 650,
20)]
Scan MILK x2: 13.00
Scan BREAD x1: 21.00
Total: 21.00
  "total": "21.00",
  "cash": "30.00",
  "change": "9.00",
Stock after sale [('APPLE', 'Apple pack', 1200, 10), ('BREAD', 'Bread', 800, 14), ('MILK', 'Milk 1L',
650, 18)]
  "cash_refund": "6.50",
Retry returns same receipt: True
Over-return rejected: Item not purchased or return quantity exceeds remaining allowance.
Final stock [('APPLE', 'Apple pack', 1200, 10), ('BREAD', 'Bread', 800, 14), ('MILK', 'Milk 1L', 650,
19)]
```

Figure 10 Actual console execution transcript

19 Verification and traceability

Local validation on 19 October 2026 ran python -m unittest discover -s tests -v from the project folder. All 18 tests passed. Tests use isolated databases; one test closes and reopens a disk database to verify persistence. This result validates the listed cases and does not establish production readiness.

The automated checks assert outcomes such as persisted stock and return allowances, rather than only reproducing implementation calculations. The multi-line insufficient-stock test confirms that a failure after one stock update rolls back earlier updates and the sale header.

Requirement / story	Test coverage and observed result
UC1 / US1, US4	Unknown product and empty sale rejected; totals, receipt values and change correct
UC1 / US2	Underpayment has no persistent effect; already-paid sale blocks another payment token
UC1 / US3	Insufficient stock rolls back all lines; cancellation leaves stock unchanged; reopen retains sale
UC2 / US5, US6	Missing receipt, unpurchased product, duplicate SKU and over-return rejected
UC2 / US6	Day 30 accepted; day 31 rejected
UC2 / US7	Partial refund uses original price after catalog change; damaged goods do not restock
UC1 and UC2 / US8	Exact payment and return replay returns saved receipt; conflicting return token rejected
Input rules	Zero, negative, fractional and Boolean quantities rejected; invalid monetary inputs rejected

20 Git evidence and submission completion

The user-created remote repository is https://github.com/denniswang12345/ProjectPos_2026. The included Git guide explains how to initialize the supplied local project, create a personal branch, commit real changes, merge with --no-ff and push main and the retained branch. No additional contributors, old commits or fabricated sprint dates should be introduced.

Git evidence must come from the actual local and remote operations. Capture git log --all --graph --decorate --oneline and git log --all --format="%h %an <%ae> %ad %s" --date=iso. Keep a separate PowerShell transcript for commands because git log records commits rather than the command history. Push the personal branch and retain it after merge as requested.

Before final submission, verify that the remote has the report, slides, source and tests, add the actual bilibili presentation URL to the cover, and include genuine Git command-history evidence. If the instructor requires recorded sprint progress, replace the not-recorded actual burndown cells with measured values and attach real review notes.

The presentation should explain the problem, cash-sale analysis, original-receipt return design, responsibilities and transaction rules, then demonstrate the program. It should use slides and live behavior, as requested by the assignment, rather than narrating the report page by page. The separate recording script provides a suggested sequence.

Individual submission and scope interpretation: the assignment labels implementation optional but also asks for source code and component evidence. This package includes a working implementation to cover both statements. The video remains an actual recording and upload task for the student.

21 References

[1] Craig Larman. Applying UML and Patterns: An Introduction to Object-Oriented Analysis and Design and Iterative Development. Third edition. Prentice Hall, 2005. Supplied textbook PDF.

Textbook reading map: Chapter 6 section 6.2 supplies the brief Handle Returns scenario; section 6.8 supplies the fully dressed Process Sale example. Chapters 9 and 10 explain domain models and SSDs. Chapter 13 explains logical layers and package diagrams. Chapters 17 and 18 cover GRASP and use-case realizations. Chapters 15 and 16 explain interaction and class diagram notation. Chapters 25 and 26 discuss further responsibility and GoF patterns.

[2] Software Engineering Practice Assignment. Supplied course assignment. Source of the required two POS use cases, agile plan, UML artifacts, repository naming and submission formats.

[3] Tech Report Cover Page. Supplied institutional cover template, retained for the report cover with student metadata updated.

[4] Software Engineering Practice Quiz1 and Quiz2. Supplied assessment question sheets. Answers are provided as separate editable documents.

[5] GitHub Docs. Adding locally hosted code to GitHub. https://docs.github.com/en/migrations/importing-source-code/using-the-command-line-to-import-source-code/adding-locally-hosted-code-to-github. Consulted 19 October 2026.

The project paraphrases and adapts the textbook methods. The 30-day receipt policy, restock decision, cash-only simplification, SQLite transaction strategy and token design are project decisions. They are not represented as requirements copied from the textbook.